

Codebase Change Summary — Week 5

Mock vs. MagicMock; Control-Flow Graphs and Coverage

SE 413/513 — Software Testing

Demo Testing Suite

What changed in `demo-testing-suite/` since Week 4, and why

1 Purpose of This Document

The `demo-testing-suite/` codebase is a single, shared project that grows one lecture at a time rather than being rebuilt from scratch each week. This document summarizes exactly what was added or changed for Week 5 and, more importantly, explains *why* each change exists — which testing concept from the two Week 5 readings it demonstrates:

- *Mock vs. MagicMock in Python* (closing out the test-doubles unit), and
- *Control-Flow Graphs and Statement, Branch, and Path Coverage*.

Read this alongside the readings and `demo-testing-suite/README.md`.

Key Idea

Nothing from Weeks 1–4 was removed or rewritten. Everything below is *additive*: the running examples you already saw are all still there, unchanged, including their two intentional `xfail` results and one intentional `FAIL`.

2 What Changed At a Glance

File	Status	Purpose
<i>Reading 1: Mock vs. MagicMock</i>		
app/accounts/user_lookup.py	New	The reading’s Examples 2–6, verbatim: functions that use their dependency through <code>len()</code> , indexing, <code>in</code> , iteration, and <code>with</code> . Ten tests: why a plain <code>Mock</code> fails, each <code>MagicMock</code> example, and the single-use-iterator pitfall.
tests/accounts/test_user_lookup.py	New	
<i>Reading 2: Control-Flow Graphs and Coverage</i>		
app/payments/transactions.py	New	The reading’s two CFG examples: <code>classify_transaction()</code> (no loops, 4 paths) and <code>contains_negative()</code> (a loop with a back edge). One test per path ($V(G) = 4$), plus the three basis paths ($V(G) = 3$) and a “many” case for the loop. <code>apply_membership_discount()</code> : the statement-vs.-branch coverage example. Both branches: the test that gives 100% statement coverage, plus the one branch coverage additionally demands. Deliberately under-tested code whose <code>pytest-cov</code> reports reproduce the reading: the 2->4 branch gap, and a Python “goto fail” analogue.
tests/payments/test_transactions.py	New	
app/pricing/membership.py	New	
tests/pricing/test_membership.py	New	
demos/coverage_gaps/	New	
<i>Tooling and documentation</i>		
requirements.txt	Modified	Adds <code>pytest-cov</code> .
docs/DEMO_GUIDE_week05_mocking_coverage.md	New	Section-by-section lecture companion for both readings.
README.md (suite and repository)	Modified	Updated layout, coverage command, expected output, and a Weekly Updates index.

Tip

Install the new dependency before running anything this week:

```
cd demo-testing-suite
pip install -r requirements.txt
```

3 Part 1: Mock vs. MagicMock

Locations: `app/accounts/user_lookup.py`, `tests/accounts/test_user_lookup.py`

Every double you wrote in Week 4 was used only through **ordinary method calls** (`repository.insert_user(...)`, `payment_service.process_payment(100)`), so `Mock` was always enough. This week's app file is the first code in the suite whose dependencies are used through **magic methods**:

Function	Operation	Magic method(s)
<code>process_users(repository)</code>	<code>len(repository), repository[0]</code>	<code>__len__, __getitem__</code>
<code>check_permission(user, p)</code>	<code>p in user.permissions</code>	<code>__contains__</code>
<code>get_names(users)</code>	<code>for user in users</code>	<code>__iter__</code>
<code>read_file(file)</code>	<code>with file as f</code>	<code>__enter__, __exit__</code>
<code>get_user_name(connection, id)</code>	<code>with connection.cursor() as c</code>	<code>__enter__, __exit__</code>

The test file walks the reading's examples in order. Three tests are worth singling out:

- `test_plain_mock_raises_type_error_on_len` and `test_plain_mock_works_once_the_magic_method_is_attached_by_hand` show the reading's core difference directly: a plain Mock fails at `len()`, until you attach `__len__` yourself, which is what `MagicMock` does for you.
- `test_check_permission_mixes_mock_and_magicmock` uses a Mock for `user` and a `MagicMock` for `user.permissions` in the same test: the choice is made *per object*, not per test.
- `test_iter_return_value_is_silently_empty_the_second_time_passes`, and that is the pitfall from the reading's Example 4 warning box: configuring `iter([...])` means the second loop quietly sees nothing.

```
pytest tests/accounts/test_user_lookup.py -v
```

Tip

Compare this file with Week 4's `tests/accounts/test_registration_service.py`. Both mock collaborators of account code, but only one needs `MagicMock`. Before reading either test, look at the *production* code and predict which. That is the reading's closing Key Idea: **let the code under test choose the mock.**

4 Part 2: Control-Flow Graphs and Path Coverage

Locations: `app/payments/transactions.py`, `tests/payments/test_transactions.py`

`app/payments/transactions.py` holds the reading's two CFG examples, verbatim, so you can draw each graph from real code and then check your hand-computed numbers against a tool. The tests are parametrized with ids that name the path each case exercises:

Function	Decisions	$V(G)$	Test ids
<code>classify_transaction</code>	3	4	<code>amount<0, intl-under-1000, domestic, intl-over-1000</code>
<code>contains_negative</code>	2	3	<code>zero-iterations, one-iteration-early-return, one-iteration-loop-exits, many-iterations-negative-late</code>

```
pytest tests/payments/test_transactions.py -v \
  --cov=app.payments.transactions --cov-branch --cov-report=term-missing
```

The report shows 100% branch coverage. For `classify_transaction()`, which has no loops, the four tests also achieve 100% *path* coverage. For `contains_negative()` they cannot: the loop's

back edge makes the number of paths grow with the input length, so the tests follow basis-path testing instead.

5 Part 2: Statement vs. Branch Coverage

Locations: `app/pricing/membership.py`, `tests/pricing/test_membership.py`, `demos/coverage_gaps/myapp.py`, `demos/coverage_gaps/test_myapp.py`

`apply_membership_discount()` appears twice, on purpose:

- In the main suite (`app/pricing/membership.py`), with both branches tested, so it reaches 100% branch coverage.
- In `demos/coverage_gaps/myapp.py`, as a four-line copy with no docstring, so its line numbers match the reading. Its test file covers *only* the member branch.

Run the demo with and without branch measurement:

```
pytest demos/coverage_gaps/test_myapp.py --cov=myapp --cov-report=term-missing
pytest demos/coverage_gaps/test_myapp.py --cov=myapp --cov-branch --cov-report=term-missing
```

The first run reports **100%**. The second reproduces the reading's report exactly:

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
-----	-----	-----	-----	-----	-----	-----
demos/coverage_gaps/myapp.py	4	0	2	1	83%	2->4

Common Pitfall

Statement coverage alone reported 100% for a function whose `False` branch was never executed. Always pass `--cov-branch`.

6 Part 2: A Python “goto fail”

Locations: `demos/coverage_gaps/handshake.py`, `demos/coverage_gaps/test_handshake.py`

The reading's case study is Apple's CVE-2014-1266, where a duplicated line made the signature check unreachable. `handshake.py` ships a Python version of the same mistake: a copy-pasted `if` that lost its `not`, so the params and signature checks can never run.

```
def verify_handshake(hash_ok, params_ok, signature_ok):
    if not hash_ok:
        return False
    if hash_ok: # KNOWN BUG -- copy-paste of the check above, lost its 'not'
        return True
    if not params_ok:
        return False
    if not signature_ok:
        return False
    return True
```

Both tests in `test_handshake.py` pass. Coverage tells a different story:

```
pytest demos/coverage_gaps/test_handshake.py \
    --cov=handshake --cov-report=term-missing
```

reports only 50% of statements executed, with lines **30–34** — every check after the bug — missing.

Key Idea

Why a copy-pasted `if`, and not simply a stray `return True`? Code after an *unconditional return* is statically dead. Python drops it when compiling, and `coverage.py` then does not count it as a statement at all, so the report would say 100%. Coverage reports can only show you lines that *could* run; to show the skipped checks, the bypass has to be conditional, as the real `goto fail` was.

7 Part 2: What Coverage Cannot Tell You

No new code: this reuses Week 3's `determine_shipping_cost()`.

```
pytest tests/pricing/test_shipping.py \
    --cov=app.pricing.shipping --cov-branch --cov-report=term-missing
```

The report shows **100% branch coverage**, and in the same output `test_valid_weights[5.0-10.0]` is still **XFAIL**. Coverage cannot see that 5.0 is the boundary that exposes the `< 5` bug; only boundary-value analysis found it.

8 Coverage on the Whole Suite

Pointing the tool at the whole codebase is a good habit, and this week it finds something:

```
pytest --cov=app --cov-branch --cov-report=term-missing
```

Every Week 5 module is at 100%. Two Week 4 files are not, for opposite reasons:

- `app/orders/order_service.py`: **80%, line 37 missing**. A *real* gap: no test drives `place_order()` down its "Payment failed" branch. It is left in place for you to close (see Suggested Next Steps).
- `app/notifications/password_reset.py`: **63%**, with `Smtplib.SMTP` missing. A *deliberate* gap: that class is the real SMTP transport, and testing it would need a real mail server, which is why the Week 4 seam exists. Chasing 100% here would make the suite worse, not better.

9 Updated: Expected Test Results

From the project root:

```
cd demo-testing-suite
pip install -r requirements.txt
pytest -v
```

you should now see **65 passed, 2 xfailed, 1 failed**: twenty more passing tests than Week 4's baseline, and the same two xfail results and one FAIL as before, unrelated to this week's additions. `demos/coverage_gaps/` is not part of that run (`pytest.ini` sets `testpaths = tests`); run it by path.

10 How This Connects to the Readings

Key Idea

Both readings ask you to look at the code under test before writing the test. The mocking reading says to let the production code's *interface* choose the double: ordinary calls mean `Mock`; `len`, indexing, `in`, iteration, and `with` mean `MagicMock`. The coverage reading says to let the production code's *structure*, its control-flow graph, show you which tests are missing, and then to remember that structure is blind to values. This week's files let you practice both against real, running code.

11 Suggested Next Steps

1. Before opening `tests/accounts/test_user_lookup.py`, read each function in `app/accounts/user_lookup.py` and decide, per object, whether you would use `Mock` or `MagicMock`.
2. Draw the CFG of `contains_negative()` and compute $V(G) = E - N + 2$. Check that it matches decisions $+1 = 3$.
3. Close the real gap: add a test to `tests/orders/test_order_service.py` whose payment stub returns `False`, then re-run the whole-suite coverage command from Section 8 and confirm `order_service.py` reaches 100%.
4. In `demos/coverage_gaps/test_handshake.py`, add the test `verify_handshake(True, True, False) is False` and watch it fail. Then fix `handshake.py`.
5. Try the readings' paper exercises (`products`, `load_config`, `grade_letter`) first by hand, then write them as code and use `--cov-branch` to check your hand-computed coverage.